

ICCS121 A3 Datalab

Mysterious Cipher

Thatchaphon Panyajamorn 6781573

Walkthrough

```
ciphertext:  
    .asciz "ddbW"          # Hardcoded ciphertext (encrypted message)
```

In the first part, we allocate memory space for the ciphertext “ddbW” using `asciz`, which is an assembler directive used to allocate space in memory for a text string and automatically append a zero byte (0x00) at the end.

```
num1:  
    .long 2  
num2:  
    .long -5  
  
    .section .text  
    .globl main
```

In the next part, we declare two long variables: `num1 = 2` and `num2 = -5`. These two variables will be used in the main function for computing a decryption key.

```
main:  
    # Load the address of ciphertext into %rdi for processing.  
    lea ciphertext(%rip), %rdi  
    movl num1(%rip), %eax  
    movl num2(%rip), %ecx  
    addl %ecx, %eax  
    movb %al, %bl
```

In the main function, we implement the “`lea`” instruction, which stands for “Load Effective Address”. This is where we compute the address of the ciphertext and put that pointer into `%rdi`.

Next, we load the value of `num1` and `num2` into `%eax` and `%ecx`, respectively. These two registers now hold 32-bit values (l = long -> 4 bytes = 32 bits). Therefore, `%eax` is now 2, and `%ecx` is now -5.

Subsequently, we perform $\%eax = \%eax + \%ecx$, which is $2 + (-5) = -3$ using `addl` instruction, then we copy low byte of $\%eax$ ($\%al$) into $\%bl$ using `movb` instruction. Therefore, $\%bl$ is now -3 (This will be our decryption key).

```
decrypt_loop:
    movzbq (%rdi), %rax
    testb %al, %al
    je decrypt_done
    subb %bl, %al
    movb %al, (%rdi)
    inc %rdi
    jmp decrypt_loop
```

After that, we will construct a loop for decryption, starting by loading the byte pointed to by $\%rdi$ (One character of the ciphertext) using `movzbq`. This instruction means “move with zero-extend, from a byte source to a quadword destination”. We used `movzbq` instead of normal `mov` because we want to move a smaller-sized source into a larger-sized destination while explicitly telling the CPU how to fill in the extra bits. In this case, `movzbq` would do.

At this point, $\%al$ = that character. Then, we check the value in $\%al$ (that character) if it encounters the null terminator or not. If true, it will jump to `decrypt_done`. If not, it will continue the loop to perform decryption process.

As it did not encounter the null terminator, $\%al$ or that character's ASCII value is increased by 3, accordingly to this expression `Subb %bl, %al` $\rightarrow \%al - (-3) \Rightarrow \%al + 3$

Lastly, we write the modified byte back into memory at the same location or basically we overwrite the ciphertext in place with the decrypted character. We will do this repeatedly until the null terminator is encountered as we advance the pointer to the next character (`inc %rdi`).

```
decrypt_done:
    # Reload the address of ciphertext to print the decrypted string.
    lea ciphertext(%rip), %rdi
    call puts          # Call external function 'puts' to display the message.

    # Exit the program.
    movl $0, %eax
    ret
...

```

Finally, the program reloads the address of the decrypted ciphertext into %rdi. Since %rdi holds the first function argument in x86-64 calling convention, “puts” receives the address of the decrypted string and prints it. We finish off our decryption process by returning the value of the decrypted characters as shown below:

Ciphertext	Plaintext
d (100)	g (103)
d (100)	g (103)
b (98)	e (101)
w (119)	z (122)

Therefore, the result is “ggez” because its ASCII value increases by 3.